



Project Data Structures CS2214

Project 3: Deque ADT

Section number. : 2

D.hanan almafriju

Team members:

1-Name :Duha Nasser Alnabati
ID : 445005974

2- Name : Tala Mazen Fadel
ID : 445007293

3-Name :Tahani Amer Albariqi
ID :445006007

4- Name : Ayan Ibrahim Alsaeedi
ID : 445007084



Interface class :

```
1 public interface Deque <E> { 1 usage 1 implementation
2
3
4 // Add an element to the front and rear
5 void addFirst(E element); //add to the front 3 usages 1 implementation
6
7 void addLast(E element); //add to the rear 2 usages 1 implementation
8
9 // remove and return
10 E removeFirst();// return front and remove it 1 usage 1 implementation
11
12 E removeLast();// return rear and remove it 1 usage 1 implementation
13
14 //return without removing
15 E first(); // return front 1 usage 1 implementation
16
17 E last(); // return rear 1 usage 1 implementation
18
19
20 boolean isEmpty(); // check if the deque is empty 6 usages 1 implementation
21
22 boolean isFull();// check if the deque is Full 3 usages 1 implementation
23
24 int size(); // return the size no usages 1 implementation
25
26 }
```



Array class:

The attributes , constructor and method isFull,isEmpty,Size

```
1 public class Array<E> implements Deque<E> { 2 usages
2
3     public static final int CAPACITY = 1000; 1 usage
4
5     public Array() { no usages
6         this(CAPACITY);
7     }
8
9     private E[] Array; 10 usages
10    private int capacity; 9 usages
11    private int size; 9 usages
12    private int front; 14 usages
13    private int rear; 13 usages
14
15    public Array(int capacity) { 2 usages
16        this.capacity = capacity;
17        this.Array = (E[]) new Object[capacity];
18        this.front = 0;
19        this.rear = -1;
20        this.size = 0;
21    }
22
23    public boolean isFull() { // This method returns true if the deque is full 3 usages
24        return size == capacity; // the number of elements (size) is equal to the maximum capacity
25    }
26
27    public boolean isEmpty() { // This method returns true if the deque is empty 6 usages
28        return size == 0;
29    }
30
```

```
    }
    public int size() { no usages
        return size;
    }
```



Array class:

method addFirst and addLast

```
public void addFirst(E element) { 3 usages
    if (isFull()) { // يشيك على المصفوفة مليانة أو لا
        throw new IllegalStateException("Deque is full");
    }
    front = ((front - 1) + capacity) % capacity;
    /*
        القوس الصغير يضمن لي أنه العنصر ينضاف في المقدمة "اليسار"
        القوس الكبير يضمن أن الاندكس ما يصير سالب لو كان اندكس الـ front صفر
        العلامة المئوية % تضمن أن المصفوفة ما حتطلع برا بتلف جوا حدودها
    */
    Array[front] = element; // أستاذ قيمة العنصر
    size++; // أزيد عدد عناصر المصفوفة بكل مرة أضيف
}

public void addLast(E element) { 2 usages
    if (isFull()) { // يشيك على المصفوفة مليانة أو لا
        throw new IllegalStateException("Deque is full");
    }
    rear = (rear + 1) % capacity;
    Array[rear] = element; // أضيف العنصر في النهاية
    size++; // أزيد حجم المصفوفة
}
```



Array class:

method remove First and Last

```
public E removeLast() { 1usage
    if (isEmpty()) { // يتحقق من المصفوفة إذا فاضية
        throw new IllegalStateException("Deque is empty"); // إذا كانت فاضية يرسل استثناء
    }
    E element = Array[rear]; // يحفظ العنصر الأخير المراد حذفه
    Array[rear] = null; // garbage collection يفرغ المكان عشان الـ
    if (front == rear) {
        front = rear = -1;
        /*
         * إذا كانوا متساويين يدل على أنه لا يوجد إلا عنصر واحد فقط فتسند القيمتين
         * إلى سالب واحد حتى تصبح المصفوفة فارغة ويحذف العنصر
         */
    } else {
        rear = (rear - 1 + capacity) % capacity; // decrement دائري
    }
    size--; // أنقص عدد العناصر
    return element; // يرجع العنصر اللي انحذف
}
```

```
public E removeFirst() { 1usage
    if (isEmpty()) { // يتحقق من المصفوفة إذا فاضية
        throw new IllegalStateException("Deque is empty"); // إذا كانت فاضية يرسل استثناء
    }
    E element = Array[front]; // يحفظ العنصر الأول المراد حذفه
    Array[front] = null; // garbage collection يفرغ المكان عشان الـ
    if (front == rear) {
        front = rear = -1;
        /*
         * إذا كانوا متساويين يدل على أنه لا يوجد إلا عنصر واحد فقط فتسند القيمتين
         * إلى سالب واحد حتى تصبح المصفوفة فارغة ويحذف العنصر
         */
    } else {
        front = (front + 1) % capacity; // increment دائري
    }
    size--; // أنقص عدد العناصر
    return element; // يرجع العنصر اللي انحذف
}
```



Array class:

method First, Last and traverse

```
public E first() { 1usage
    if (isEmpty()) { // يتحقق من المصفوفة إذا فاضية
        throw new IllegalStateException("Deque is empty"); // إذا كانت فاضية يرسل استثناء
    }
    return Array[front]; // يرجع أول عنصر
}

public E last() { 1usage
    if (isEmpty()) { // يتحقق من المصفوفة إذا فاضية
        throw new IllegalStateException("Deque is empty"); // إذا كانت فاضية يرسل استثناء
    }
    return Array[rear]; // يرجع آخر عنصر
}

public void traverse() { 2 usages
    if (isEmpty()) { // يشيك لو كانت المصفوفة فارغة
        System.out.println("The Array is empty.");
        return;
    }
    System.out.print("Elements in the Array: ");
    int current = front; // يبدأ من المؤشر الأمامي
    for (int i = 0; i < size; i++) {
        System.out.print(Array[current] + " "); // يطبع العنصر الحالي
        current = (current + 1) % capacity; // انتقل إلى العنصر التالي بشكل دائري
    }
    System.out.println();
}
}
```

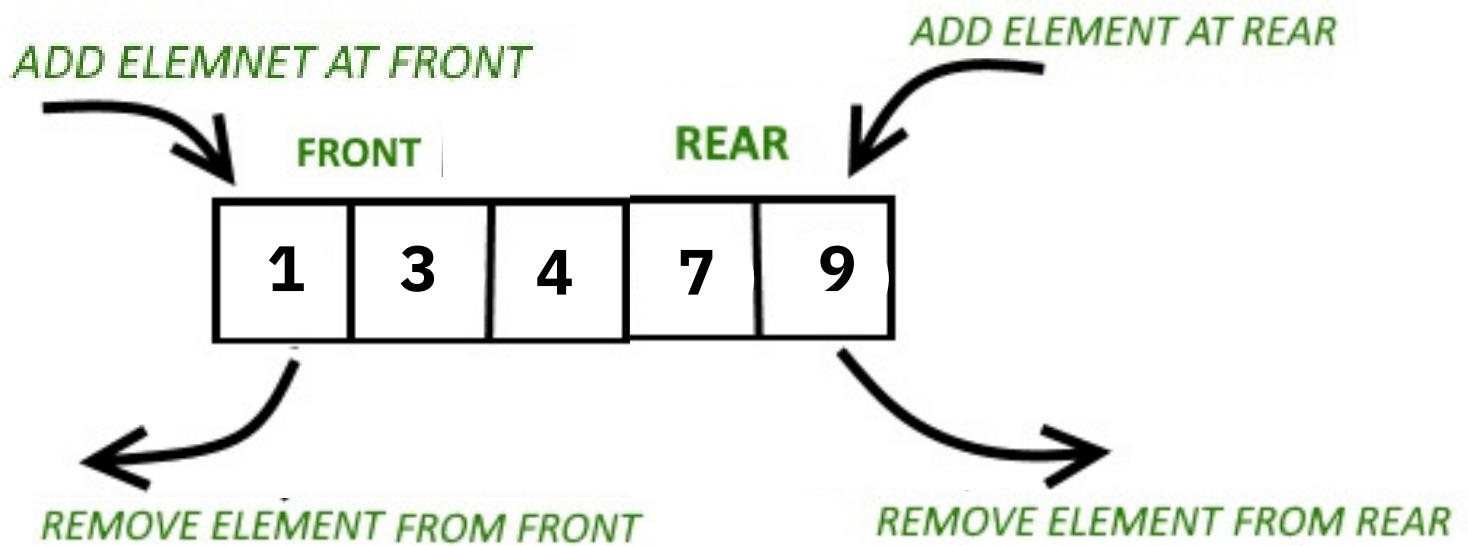


Main class :

```
1 public class Main {
2     public static void main(String[] args) {
3         Array<Integer> array = new Array<>( capacity: 5);
4
5         // إضافة عناصر إلى المصفوفة
6         array.addLast( element: 7);
7         array.addFirst( element: 4);
8         array.addFirst( element: 3);
9         array.addFirst( element: 1);
10        array.addLast( element: 9);
11
12        System.out.println("All Elements after additions:");
13        array.traverse(); // عرض جميع العناصر
14
15        System.out.println("Is the Array Full? " + array.isFull());
16
17        // عرض أول وآخر عنصر
18        System.out.println("First Element: " + array.first());
19        System.out.println("Last Element: " + array.last());
20
21        // إزالة العناصر وعرض الحالة بعد كل عملية
22        System.out.println("Remove The Last Element and The first Element: " + array.removeLast()+" ,"+ array.removeFirst());
23        System.out.println("All Elements after removing:");
24        array.traverse(); // عرض جميع العناصر
25
26        // التحقق إذا كانت المصفوفة فارغة
27        System.out.println("Is the Array Empty? " + array.isEmpty());
28    }
29 }
```



The illustrations:





Output :

```
C:\Users\duhan\.jdk\openjdk-21.0.2\bin\java.exe "-javaagent:C:\Program Files\JetBrain
All Elements after additions:
Elements in the Array: 1 3 4 7 9
Is the Array Full? true
First Element: 1
Last Element: 9
Remove The Last Element and The first Element: 9 ,1
All Elements after removing:
Elements in the Array: 3 4 7
Is the Array Empty? false

Process finished with exit code 0
```